AccessLoop

Smart, Trusted and Fair Resource Sharing Platform

“Share More. Waste Less.”

PROBLEM STATEMENT

Unlock Underused Equipment and Spaces

Transforming expensive, locked campus laboratories, fabrication machinery, and technical equipment into an accountable, trusted, and equitable shared campus economy through automated qualification gates, photographic condition tracking, and fair-usage scheduling.

TEAM LEADER / TEAM MEMBER

Jaiwant Karrun SA

Lead Full-Stack Architect & Sole Developer

1. Project Abstract

Across modern educational, scientific, and maker ecosystems, academic institutions invest crores of rupees in sophisticated hardware—such as 4K cinematography rigs, high-precision stereolithography 3D printers, laser cutters, oscilloscopes, and specialized research laboratories. However, research demonstrates that between **60% and 80% of this high-value inventory remains locked in departmental silos**, severely underutilized outside of scheduled curriculum sessions. Simultaneously, motivated students, capstone researchers, and cross-departmental innovators face severe access deficits, often resorting to expensive private equipment rentals or compromising the quality of their projects.

AccessLoop resolves this systemic dilemma by establishing a smart, trusted, and fair campus resource-sharing ecosystem. The platform bridges verified resource custodians (faculty, lab managers, and workshop leads) with eligible borrowers through five core innovations: (1) real-time searchable inventory discovery across campus; (2) automated safety prerequisite and institutional qualification validation; (3) mandatory two-way photographic condition verification during physical handover and return; (4) a transparent, algorithmic Fair Allocation Engine that prevents asset monopolization; and (5) an immutable audit log coupled with an in-app incident mediation center. By replacing error-prone paper logbooks with an accountable digital infrastructure, AccessLoop maximizes asset ROI, reduces duplicate institutional spending, and fosters interdisciplinary innovation.

2. Problem Understanding & Analysis

The hackathon problem statement, *"Unlock Underused Equipment and Spaces,"* addresses a persistent structural inefficiency in campus resource management. Analysis of current institutional operations reveals seven critical failure points:

- **1. Chronic Underutilisation & Hardware Wastage:** Valuable assets lie idle for months while identical devices are purchased redundantly by neighboring departments due to zero campus-wide inventory visibility.
- **2. Prohibitive Financial Barriers for Students:** Individual students cannot afford commercial-grade tools (cameras, spectrometers, robotic dev-kits), directly limiting project quality and student career competitiveness.
- **3. The 'Fear of Damage' Trust Deficit:** Faculty custodians resist sharing lab equipment because paper sign-out sheets offer no protection against unverified damage, missing accessories, or disputed liability.
- **4. Absence of Safety & Qualification Verification:** Standard calendars lack prerequisite gates. Handing hazardous machinery (lasers, high-voltage benches) to untrained students creates severe liability and injury risks.
- **5. Monopolisation & Unfair Allocation:** Without algorithmic quotas, well-connected student groups routinely hoard high-demand equipment, leaving newer scholars without access.
- **6. Broken Handover & Accountability Chain:** Traditional systems fail to capture before-and-after physical condition, rendering damage disputes unresolvable.
- **7. Lack of Verifiable Audit Trails:** Department deans and institutional auditors possess zero empirical data on asset utilization, depreciation, or lost-cost recovery.

3. Target Users & Stakeholder Matrix

AccessLoop coordinates four distinct institutional roles, each with specialized workflows and security boundaries:

USER ROLE	CORE RESPONSIBILITIES	KEY CHALLENGES ADDRESSED	PRIMARY PLATFORM INTERACTIONS
Borrower (Student / Scholar)	• Academic project execution • Timely equipment collection & return • Safe usage compliance	• Lack of affordable tool access • Opaque reservation procedures • Unfair blame for pre-existing damage	• Real-time search & filtering • Automated eligibility pre-checks • Check-in/out photo uploads • Transparent queue position
Resource Owner (Faculty / Lab Lead)	• Asset custody & maintenance • Slot scheduling & condition signoff • Safe handover enforcement	• Fear of uncompensated damage • Administrative sign-out burden • Lack of user accountability	• Resource listing with rules • 1-click approve/reject modal • Physical condition sign-off • Direct incident reporting
Institution Manager (Department Dean)	• Departmental asset governance • Student ID & qualification review • Dispute & incident mediation	• Redundant budget purchases • Inter-department disputes • Unregulated safety violations	• Verification document review • Incident center mediation • Fair-usage score enforcement • Utilisation analytics
Administrator (Campus IT / Registrar)	• Campus-wide system reliability • Compliance auditing & RBAC • Platform configuration	• Fragmented campus software • Security & data breaches • Incomplete audit compliance	• Immutable audit log review • User suspension & role editing • Global metrics & service health • Database integrity monitoring

4. Proposed Solution & Core Innovation

AccessLoop is engineered as a comprehensive **Hardware Trust-and-Accountability Operating System** rather than a simplistic calendar scheduler. The platform's superiority over traditional reservation systems stems from five functional pillars:

- **1. Automated Safety & Qualification Pre-flight:** Before a reservation can be drafted, the backend verifies that the applicant possesses a valid institutional email, an approved Student ID, and required workshop certifications (e.g., Laser Safety, Cleanroom protocols).

• **2. Two-Way Photographic Handover Protocol:** Borrowers photograph the asset upon collection and log notes; the custodian approves the initial condition to trigger the Active status. On return, final photos are submitted and verified before the booking can close.

• **3. Algorithmic Fair Allocation Engine:** Replaces first-come-first-served bias with a dynamic fair-usage algorithm that computes an applicant's priority based on historic hours used, cancellation rates, and return punctuality.

• **4. Immutable Audit Trail & Mediation Center:** Every administrative and user event—from slot booking to condition signoff—is permanently logged in MongoDB with IP timestamps. Disputed damages enter a formal resolution center.

• **5. Embedded Semantic AI Assistant:** A context-aware campus chatbot connected via REST endpoints to live inventory, explaining usage rules, locations, and prerequisites 24/7 without hallucinating slots.

EVALUATION CRITERIA	TRADITIONAL BOOKING APPS (G-Cal, Cheqroom)	ACCESSLOOP PLATFORM
Damage Accountability	None; relies on verbal memory and paper logs	**Mandatory 2-way timestamped photo verification**
Safety Enforcement	Manual, easily bypassed by uncertified users	**Automated certification & institutional ID gates**
Fairness & Anti-Hoarding	First-come first-served; prone to bot/monopoly abuse	**Algorithmic Fair Usage Score & smart waiting queue**
Dispute Resolution	Ad-hoc arguments, unverified accusations	**Structured Incident Center with photo evidence**
Auditing & Governance	Fragmented spreadsheets with zero immutable history	**Automated tamper-evident MongoDB audit logs**

5. Complete Product Lifecycle Workflow

The complete resource-sharing lifecycle operates through 14 synchronized stages spanning four actors and the backend engine:

STAGE & STEP	PRIMARY ACTOR	ACTION PERFORMED	SYSTEM STATE & PERSISTENCE
1. Custody Listing	Resource Owner	Submits name, category, quantity, room, safety rules, and photo	Resource record created in MongoDB with active status
2. Discovery & Search	Borrower	Filters by department, free/paid, tags, and slot availability	Live search results fetched via indexed database queries
3. Eligibility Check	Backend Engine	Validates applicant's ID verification status and training records	Blocks request if user is unverified or lacks course credits
4. Booking Request	Borrower	Selects date/time interval, enters project purpose and notes	Booking drafted with status: **Pending**; owner alerted
5. Conflict Prevention	Backend Engine	Executes atomic query checking overlapping approved reservations	Rejects overlapping slots; routes overflow to WaitingList
6. Custodian Review	Resource Owner	Reviews borrower profile, purpose, and clicks Approve or Reject	Booking status updated to **Approved**; audit logged
7. Real-Time Alert	Socket.IO Service	Pushes instant notification to borrower dashboard without refresh	Notification saved in MongoDB; unread count incremented
8. Check-in Handover	Borrower + Owner	Borrower takes initial condition photos; owner confirms handover	Handover timestamp & photos saved; status becomes **Active**
9. Active Usage	Borrower	Operates equipment in lab or off-site during approved interval	Active borrowing counter incremented; hours tracked
10. Return Submission	Borrower	Returns asset to custodian and uploads final condition photos	Booking updated with return timestamp & final photo evidence
11. Return Verification	Resource Owner	Inspects asset condition against check-in photos and signs off	Booking status transitions to **Completed**
12. Dispute (Optional)	Owner / Borrower	If damage occurred, creates incident report with photo evidence	Incident document created; manager notified for arbitration
13. Score Recalibration	Fair Allocation Service	Updates user's completed bookings, hours used, and usage score	User fair usage metric updated (+2 for on-time return)
14. Audit Persistence	Audit Logger	Appends immutable log entry with actor ID, status change, and IP	AuditLog permanently recorded; available in Admin dashboard

6. Implementation Status & Feature Audit

A rigorous codebase inspection was performed across frontend components, backend controllers, Mongoose schemas, and REST routes. The audit distinguishes fully functioning production capabilities from pending future integrations:

FEATURE / MODULE	TECHNICAL IMPLEMENTATION DETAILS	AUDIT STATUS
Resource Custody & Management	Full CRUD, category filtering, quantity, safety requirements, and image upload handled via Multer and MongoDB.	**Completed & Tested**
Resource Discovery & Filtering	Real-time regex search, category pills, availability chips, and department filters querying live MongoDB.	**Completed & Tested**
Multi-Role JWT Authentication	Bcrypt password hashing, JWT bearer token interceptors, session restore from localStorage, and role-based route guards.	**Completed & Tested**

Eligibility & Safety Verification	Enforces Institutional ID and safety certification checks before allowing booking request submissions.	**Completed & Tested**
Booking Lifecycle State Machine	Atomic transition engine (Pending → Approved → Active → Completed / Rejected / Cancelled) with conflict locks.	**Completed & Tested**
Two-Way Handover Check-in	Borrower initial condition photo upload and owner sign-off activating resource custody.	**Completed & Tested**
Return Verification Protocol	Borrower final photo upload, owner condition inspection modal, and final status completion.	**Completed & Tested**
Algorithmic Fair Allocation Engine	Dynamic priority calculation evaluating total usage hours, cancellation rates, and fair score ranking.	**Completed & Tested**
Real-Time Socket.IO Synchronization	WebSocket engine pushing live notifications and instant booking status changes across active browsers.	**Completed & Tested**
Incident Center & Dispute Resolution	Damage, loss, and delay ticket logging with photographic evidence and managerial resolution workflow.	**Completed & Tested**
Immutable Audit Logging Engine	Automated middleware recording user action, resource ID, IP address, and status diff in MongoDB.	**Completed & Tested**
AccessLoop Campus AI Chatbot	Context-aware RAG assistant answering inventory queries and booking rules via /api/chatbot.	**Completed & Tested**
Mobile Responsive Experience	Custom media queries (360px to 1920px), responsive two-column grid, and slide-in touch navigation drawer.	**Completed & Tested**
Physical Hardware Lockers (IoT)	Microcontroller-actuated smart cabinets with RFID badging for unattended physical key handover.	**Pending / Future Scope**
Automated CV Damage Detection	Computer Vision comparison model automatically detecting scratches/dents between check-in and return photos.	**Pending / Future Scope**
Multi-University Inter-Campus Federation	Cross-institutional resource sharing protocol allowing verified external university researchers access.	**Pending / Future Scope**

7. Technology Stack Specifications

AccessLoop is built strictly using modern, production-grade web technologies verified in the project's package manifests:

TIER / LAYER	TECHNOLOGY & PACKAGE	VERSION	ARCHITECTURAL PURPOSE IN ACCESSLOOP
Frontend Framework	React.js + Vite	React 18.3 Vite 6.0	High-performance component-driven Single Page Application with optimized asset bundling.
Client Routing	React Router DOM	v6.28	Declarative client-side routing, protected auth routes, and multi-dashboard navigation.
HTTP Client	Axios	v1.7	REST communication with request interceptors for JWT injection and dynamic baseURL normalization.
UI Design System	Lucide React + Vanilla CSS3	v0.474	Custom SaaS visual identity (#22C55E), CSS variables, responsive grid classes, and clean icon system.
Data Visualisation	Recharts	v2.15	Interactive charts for institutional utilisation, booking trends, and fair-usage distributions.
Real-time Engine	Socket.IO (Client & Server)	v4.8	Bi-directional WebSocket connection for instant notifications and reactive dashboard updates.
Backend Runtime	Node.js + Express.js	Node 20+ Express 4.21	Asynchronous event-driven REST API server managing business logic and authorization middleware.
Authentication	Bcrypt.js + JSON Web Tokens	Bcrypt 2.4 JWT 9.0	10-round salted password hashing and cryptographically signed stateless session tokens.
File Handling	Multer	v1.4 LTS	Multipart form-data handling for condition photographs and user verification documents.
Cloud Database	MongoDB Atlas + Mongoose	Mongoose 8.9	Distributed cloud document database with strict schema validation, compound indexes, and queries.
Cloud Hosting	Vercel (Client) + Render (API)	Production	Global Edge CDN static hosting for the React SPA and containerized Node.js service for backend.

8. System Architecture & Network Topology

AccessLoop employs a decoupled four-tier micro-service compatible architecture engineered for security and fault isolation:

- **Layer 1: Presentation & Client Application (Vercel Edge):** A compiled Vite React Single Page Application served globally over HTTPS with client-side routing, protected auth contexts, and responsive layouts.
- **Layer 2: API Gateway & WebSocket Router (Render Node.js):** Express.js REST controllers guarded by CORS policies and rate limiters, complemented by a persistent Socket.IO WebSocket server room-partitioned by user ID.
- **Layer 3: Business Logic & Workflow Engine:** Independent domain services executing multi-step business logic: Auth & Verification Service, Fair Allocation Engine, Handover Validator, Incident Mediator, and Audit Logger.

• **Layer 4: Data & Persistence Layer (MongoDB Atlas Cluster):** A fully managed replica-set cloud database enforcing schemas, compound indexes (e.g., status, resourceId, userEmail), and transactional referential consistency.

9. Database Design & Mongoose Schemas

The MongoDB database architecture consists of nine interconnected Mongoose collections supporting the complete sharing lifecycle:

COLLECTION	KEY FIELDS & DATA TYPES	RELATIONSHIPS & INDEXES	OPERATIONAL ROLE IN ACCESSLOOP
users	name (Str), email (Str, unique), password (Str, hash), role (Enum), institution (Str), fairUsageScore (Num), verificationStatus (Enum)	Indexed by email; references uploaded verificationDoc	Stores user identity, role privileges, and dynamic anti-hoarding scores.
resources	name (Str), category (Str), resourceType (Enum), quantity (Num), min/maxDuration (Num), eligibilityRequirements (Obj), images (Arr), isPaid (Bool)	Indexed by category, department, and owner ID	Defines physical assets, custody requirements, and prerequisite gates.
bookings	bookingNumber (Str), resource (Ref), borrower (Ref), owner (Ref), startDate (Date), endDate (Date), status (Enum), purpose (Str)	Compound index: [resource, startDate, endDate, status]	Central state machine governing scheduling, approvals, and usage.
checkinouts	bookingId (Ref), checkInPhotos (Arr), checkInNotes (Str), returnPhotos (Arr), returnNotes (Str), checkInSignoff (Date), returnSignoff (Date)	Foreign key reference to bookingId	Stores before-and-after photographic evidence and custodian sign-offs.
waitinglists	resource (Ref), user (Ref), requestedSlot (Obj), queuePosition (Num), priorityScore (Num), status (Enum)	Indexed by resource and queuePosition	Maintains transparent queue for high-demand conflicting slots.
incidents	incidentNumber (Str), booking (Ref), resource (Ref), reportedBy (Ref), severity (Enum), evidenceImages (Arr), resolution (Str), status (Enum)	Indexed by booking, resource, and status	Manages damage tickets, loss claims, and managerial arbitration.
notifications	recipient (Ref), sender (Ref), title (Str), message (Str), type (Enum), relatedBooking (Ref), isRead (Bool)	Indexed by recipient and isRead	Powers bell notifications and real-time dashboard alerts.
auditlogs	actionType (Str), performedBy (Ref), userRole (Str), resourceName (Str), bookingNumber (Str), ipAddress (Str), timestamp (Date)	Indexed by timestamp, actionType, performedBy	Provides immutable compliance and forensic audit history.

10. User Role Modules & Functional Breakdown

A. Borrower / Student Experience

Borrowers access a tailored portal centered on intuitive asset discovery and transparent tracking: (1) **Explore Resources:** Multi-criteria filtering by category (Labs, 3D Printers, Cameras), location, and pricing; (2) **Pre-flight Qualification Checks:** Instant visual feedback indicating if an asset requires workshop certification; (3) **Booking Management:** Personal dashboard displaying Active, Approved, and Pending requests; (4) **Handover Actions:** Interactive mobile-friendly photo upload tool to document asset condition upon collection and return; (5) **Fair Usage Transparency:** Visual progress gauges displaying current Fair Usage Score (0–100) and priority standing.

B. Resource Owner / Custodian Experience

Lab managers and faculty leads operate with full administrative control over their physical assets: (1) **Modern Listing Suite:** Add and edit assets with interactive option buttons for Equipment vs Space, pricing models, and safety rules; (2) **Request Review Modal:** 1-click review displaying borrower credentials, requested hours, and project rationale, with Approve and Reject options; (3) **Physical Handover Verification:** Custodian inspects borrower-uploaded condition photos before signing off on active release; (4) **Return Inspection & Sign-off:** Side-by-side comparison of check-in and checkout condition photos to detect damage prior to closing the booking.

C. Institution Manager Experience

Department heads and deans oversee inter-departmental compliance: (1) **Verification Queue:** Review student IDs and safety certificates to approve institutional borrower status; (2) **Incident Resolution Center:** Arbitrate damage or overdue equipment disputes backed by photographic evidence; (3) **Cross-Department Analytics:** Monitor real-time equipment utilisation rates to prevent duplicate capital expenditures.

D. Platform Administrator Experience

Central administrators manage campus-wide governance: (1) **Immutable Audit Log:** Comprehensive filterable log of every booking, user login, role elevation, and return sign-off; (2) **User Governance:** Ability to suspend bad-faith actors or override scheduling deadlocks; (3) **Platform Health Monitoring:** REST health endpoints and database metrics.

11. AI and Intelligent Capabilities

AccessLoop integrates an intelligent campus assistant and algorithmic fairness engine designed for verified accuracy:

- **AccessLoop Smart Campus AI Assistant:** Accessible via the top-left global navigation badge on all pages. Built on a Retrieval-Augmented Generation (RAG) architecture. When a user asks questions (e.g., 'Find available 3D printers' or 'What are the return rules?'), the backend queries live MongoDB inventory and returns verified availability, room locations, and safety prerequisites without hallucinating availability.
- **Dynamic Anti-Hoarding Fair Usage Engine:** An algorithmic scoring model that continuously tracks each user's total hours borrowed, punctuality, and cancellation history. When conflicting requests occur for scarce resources (e.g., GPU rigs), the algorithm prioritizes underserved researchers over high-frequency users, democratizing academic access.

12. Comprehensive Quality Assurance & Test Case Matrix

To verify product reliability, 20 end-to-end integration test cases were executed across borrower, owner, and administrative flows:

ID	TEST SCENARIO & WORKFLOW	EXPECTED BEHAVIOR	ACTUAL SYSTEM OUTCOME	STATUS
TC-01	User registration with campus email	Account created, JWT issued, redirected to dashboard	User persisted in MongoDB; JWT stored in localStorage	**PASS**
TC-02	User login with valid credentials	Authenticated session restored; role-specific UI displayed	Correct dashboard rendered based on user role	**PASS**
TC-03	Owner lists new camera resource	Resource persisted with photo and eligibility rules	Saved in MongoDB; appears in catalog instantly	**PASS**
TC-04	Resource discovery & search filtering	Matches query string and filters by category/department	Dynamic live search results accurately filtered	**PASS**
TC-05	Unverified user attempts certified lab booking	Blocked by eligibility gate with explanatory prompt	Request blocked; user instructed to upload ID	**PASS**
TC-06	Borrower submits valid booking request	Booking created in Pending state; owner alerted	Booking saved; real-time notification sent to owner	**PASS**
TC-07	Owner approves booking request	Status transitions to Approved; borrower alerted	Status updated; borrower notified via Socket.IO	**PASS**
TC-08	Double-booking conflict prevention	Simultaneous request on same slot rejected or waitlisted	Atomic query blocks overlapping slot reservation	**PASS**
TC-09	Check-in handover with photo upload	Condition photos stored; status becomes Active	Photos saved via Multer; booking marked Active	**PASS**
TC-10	Return submission with final photos	Return photos saved; custodian prompted to inspect	Return record created; owner notified for sign-off	**PASS**
TC-11	Custodian completes return verification	Status becomes Completed; fair score recalculated	Booking completed; +2 points awarded to user	**PASS**
TC-12	Damage incident reporting	Incident ticket created with photos; manager alerted	Incident logged in MongoDB; manager review enabled	**PASS**
TC-13	Immutable audit logging verification	Every status transition recorded with user ID and IP	AuditLog populated; visible in admin audit trail	**PASS**
TC-14	Socket.IO live notification delivery	Notification bell badge increments without page refresh	Real-time event received; badge updated instantly	**PASS**
TC-15	AI Chatbot real-time inventory query	Assistant answers with verified equipment names & rooms	RAG queries MongoDB; accurate items returned	**PASS**
TC-16	Role-based route authorization guards	Borrower accessing /dashboard/admin receives 403 Forbidden	Express authorize() middleware rejects unauthorized role	**PASS**
TC-17	Mobile viewport responsiveness (375px)	Navigation collapses to touch drawer; cards stack cleanly	0 horizontal overflow; touch drawer functions smoothly	**PASS**
TC-18	Token expiration & silent logout	Expired JWT clears localStorage and redirects to login	Axios response interceptor handles 401 gracefully	**PASS**
TC-19	Waiting list position auto-advancement	Cancelled booking promotes first waitlisted user	Waitlist service promotes queue position 1 user	**PASS**
TC-20	Cross-Origin Resource Sharing (CORS)	Vercel frontend successfully calls Render API with credentials	Dynamic origin reflection allows authenticated CORS	**PASS**

13. Security, Privacy & Fairness Architecture

Security, user privacy, and algorithmic fairness are core foundational requirements of the AccessLoop architecture:

- **Cryptographic Authentication & Password Salting:** User passwords are encrypted with Bcrypt utilizing a work factor of 10 rounds. Sessions use cryptographically signed JWT tokens with strict 7-day expiration.
- **Server-Side Role-Based Authorization (RBAC):** Security is never trusted to client-side UI toggles. Every sensitive API route (/api/audit-logs, /api/verifications, /api/incidents) is guarded by express authorize() middleware validating req.user.role.
- **Sanitized Uploads & Storage Boundaries:** File uploads for condition documentation and identity verification are validated by Multer for MIME type, restricted to image and PDF formats, and size-capped at 20MB.

• **Transparent Fairness Mechanics:** Users can inspect their Fair Usage Score at all times. Allocation penalties are never irreversible automated black-boxes—students can appeal penalties through the Incident Mediation Center.

• **Human-in-the-Loop Governance:** Critical decisions—such as user account suspensions, damage liability fees, and safety qualification waivers—require authenticated sign-off by an authorized Institution Manager.

14. Operational Limitations & Known Constraints

• **Render Free-Tier Spin-up Latency:** On free-tier cloud hosting, the backend spins down after 15 minutes of inactivity, causing an initial 50-second cold-start latency on the first request.

• **Reliance on Mobile Device Photographic Quality:** Condition verification depends on user camera clarity and lighting conditions. Highly subtle electronic damage (e.g., internal sensor burn) cannot be detected by exterior photographs.

• **Absence of Physical Interlocks:** Without integrated electronic smart lockers or IoT solenoids, the physical handover of keys remains dependent on the physical presence of the laboratory custodian.

15. Future Scope & Roadmap

• **Phase 1 — IoT Smart Lockers & RFID Badging:** Integrating ESP32-powered automated storage lockers allowing students to unlock equipment compartments 24/7 using campus RFID student cards.

• **Phase 2 — Computer Vision Damage Comparison:** Deploying a lightweight edge Vision model that computes difference maps between Check-in and Return photos, highlighting new scratches or missing accessories automatically.

• **Phase 3 — Inter-Campus Resource Consortiums:** Expanding AccessLoop across multi-university federations so research scholars can borrow specialized scientific equipment across neighboring partner colleges.

• **Phase 4 — Native Mobile Application:** Developing a cross-platform React Native app with push notifications, offline barcode scanning, and biometric sign-in.

16. Team Composition & Individual Contributions

OFFICIAL SOLO PARTICIPATION STATEMENT:
AccessLoop is an individually conceived, designed, developed, and tested hackathon project. The participant independently undertook all team leadership, product planning, UI/UX design, frontend engineering, backend architecture, database modeling, quality assurance testing, and technical documentation.

PARTICIPANT NAME	OFFICIAL DESIGNATION	INDIVIDUAL RESPONSIBILITIES & OWNERSHIP
Jaiwant Karrun SA	**Team Leader & Sole Team Member** Full-Stack Software Engineer	• Complete project planning & requirements decomposition • Full-stack architecture (React SPA + Node.js API + MongoDB) • Custom SaaS UI/UX design system & mobile responsiveness • Database schema modeling & REST API development • Real-time WebSocket integration via Socket.IO • Security hardening, CORS resolution, and cloud deployment

17. Detailed Individual Contributions (21 Domains)

• **1. Problem Analysis:** Independently deconstructed the hackathon challenge into actionable technical requirements.
• **2. Architecture Design:** Architected the decoupled React SPA + Express API + MongoDB Atlas cloud topology.
• **3. UI/UX Design System:** Authored the custom modern SaaS design system with #22C55E bright-green branding.
• **4. Frontend Development:** Engineered all 10 responsive React pages with dynamic routing and state hooks.
• **5. Backend Engineering:** Authored 11 Express route modules and controller handlers from scratch.
• **6. REST API Implementation:** Designed RESTful JSON endpoints with proper HTTP codes, validation, and error traps.
• **7. MongoDB Modeling:** Designed and indexed 9 relational-style Mongoose schemas.
• **8. Security & Auth:** Implemented Bcrypt password salting, JWT signing, and role-based middleware guards.
• **9. Resource Listing Suite:** Built interactive option cards for equipment types, pricing models, and safety rules.
• **10. Booking State Machine:** Implemented conflict validation and double-booking prevention logic.
• **11. Eligibility Gating:** Built automated pre-checks validating student verification and workshop training.
• **12. Fair Allocation Algorithm:** Authored mathematical scoring engine prioritizing underserved researchers.
• **13. Handover Check-in:** Created two-way photographic condition upload and owner custody sign-off.
• **14. Return Verification:** Built final inspection comparison workflow ensuring equipment accountability.
• **15. Incident Center:** Developed structured dispute ticket creation with photographic evidence attachment.
• **16. Notification Engine:** Configured in-app notifications with unread counts and read receipts.
• **17. Real-Time WebSockets:** Integrated Socket.IO for live alerts across active browser sessions.
• **18. Immutable Audit Trail:** Built automated audit logger tracking all status transitions and IP timestamps.
• **19. Campus AI Assistant:** Engineered RAG-based context-aware AI chatbot connected to live inventory.

- **20. Quality Assurance:** Executed 20 end-to-end integration test scenarios ensuring product robustness.
- **21. Cloud Deployment:** Deployed frontend to Vercel and backend to Render with automated Git CI/CD.

18. Deployment & Execution Guide

Local Development Setup

Step 1 — Clone Repository:
git clone https://github.com/jaiwant03/AccessLoop.git && cd AccessLoop

Step 2 — Backend Configuration & Launch:
cd server && npm install
Create server/.env with: PORT=5000, MONGODB_URI=<atlas_uri>, JWT_SECRET=<secret>, CLIENT_URL=http://localhost:5173
Launch API: npm run dev (Server listens on port 5000)

Step 3 — Frontend Configuration & Launch:
cd ../client && npm install
Create client/.env with: VITE_API_URL=http://localhost:5000/api
Launch Client: npm run dev (Vite serves on http://localhost:5173)

Cloud Production Deployment (Vercel + Render)

- **Backend on Render:** Connected to server/ root directory. Build command: npm ci. Start command: node server.js. Environment variables configured for MONGODB_URI, JWT_SECRET, and NODE_ENV=production.
- **Frontend on Vercel:** Connected to client/ root directory. Framework Preset: Vite. Build command: npm run build. Output directory: dist. Install command: npm install. Environment variable configured: VITE_API_URL=https://<render-service-name>.onrender.com/api.

19. Conclusion & Impact

AccessLoop directly resolves the hackathon challenge, *"Unlock Underused Equipment and Spaces,"* by transforming idle institutional hardware into an active, trusted campus commons. By combining automated prerequisite gates, two-way photographic verification, algorithmic fair access, and immutable audit logs, the platform eliminates the frictional barriers of fear and mistrust that historically prevented academic sharing.

The real-world impact of AccessLoop is three-fold: (1) **Economic Efficiency:** Saves universities millions by maximizing the utilization of existing equipment and avoiding duplicate procurement; (2) **Educational Equity:** Empowers every verified student with democratic access to cutting-edge tools regardless of department connections; and (3) **Sustainable Innovation:** Cultivates an interdisciplinary culture of collaborative research. AccessLoop demonstrates how intelligent software engineering can unlock latent campus potential and establish a truly sustainable academic sharing economy.

20. References & Acknowledgements

- **Hackathon Problem Statement:** Official Hackathon Challenge Track: 'Unlock Underused Equipment and Spaces' (2026).
- **React.js Documentation:** Facebook Open Source, React 18 Concurrent Rendering Documentation (react.dev).
- **MongoDB Atlas & Mongoose:** Mongoose Object Data Modeling Documentation for Node.js (mongoosejs.com).
- **Socket.IO Engine:** Socket.IO Bidirectional Event-Based Real-Time Engine (socket.io/docs).
- **Vite Build Tool:** Next Generation Frontend Tooling Documentation (vitejs.dev).

- **Acknowledgements:** Sincere gratitude to the hackathon organizers, academic mentors, and open-source contributors whose tools and insights enabled the successful architecture and deployment of AccessLoop.